

WEEK 17.1 AI ASSISTED

T. VIKAS

2303A52281

BATCH 43

TASK1:

PROMPT: "Generate a Python script using Pandas to clean a customer dataset by handling missing values (age, income, city), removing duplicates, standardizing text fields, and encoding categorical variables like gender and city. Ensure the output is a cleaned dataframe ready for analysis."

CODE:

```
week 17.1.py > ...
1  import pandas as pd
2
3  # Load dataset
4  data = pd.read_csv("shopping_trends.csv")
5
6  # Display original data
7  print("Original Data:")
8  print(data.head())
9
10 # -----
11 # 1. Handle Missing Values
12 # -----
13
14 # Fill missing numerical values with median
15 if 'Age' in data.columns:
16     data['Age'].fillna(data['Age'].median(), inplace=True)
17
18 if 'Income' in data.columns:
19     data['Income'].fillna(data['Income'].median(), inplace=True)
20
21 # Fill missing categorical values with mode
22 if 'City' in data.columns:
23     data['City'].fillna(data['City'].mode()[0], inplace=True)
24
```

```

33 if 'City' in data.columns:
34     data['City'] = data['City'].str.lower().str.strip()
35
36 if 'Gender' in data.columns:
37     data['Gender'] = data['Gender'].str.lower().str.strip()
38
39 # -----
40 # 4. Encode Categorical Variables
41 # -----
42
43 # One-hot encoding for City
44 if 'City' in data.columns:
45     data = pd.get_dummies(data, columns=['City'], drop_first=True)
46
47 # Label encoding for Gender
48 if 'Gender' in data.columns:
49     data['Gender'] = data['Gender'].map({'male': 0, 'female': 1})
50
51 # -----
52 # Final Output
53 # -----
54 print("\nCleaned Data:")
55 print(data.head())
56

```

```

56
57 # Save cleaned dataset
58 data.to_csv("cleaned_shopping_trends.csv", index=False)

```

Output:

```

PS C:\Users\vikas\Downloads\AI Assist Coding> python week 17.1.py
Original Data:
  Customer ID  Age  ... Preferred Payment Method Frequency of Purchases
0           1   55  ...                Venmo      Fortnightly
1           2   19  ...                Cash      Fortnightly
2           3   50  ...          Credit Card        Weekly
3           4   21  ...             PayPal        Weekly
4           5   45  ...             PayPal        Annually

[5 rows x 19 columns]

Cleaned Data:
  Customer ID  Age  ... Preferred Payment Method Frequency of Purchases
0           1   55  ...                Venmo      Fortnightly
1           2   19  ...                Cash      Fortnightly
2           3   50  ...          Credit Card        Weekly
3           4   21  ...             PayPal        Weekly
4           5   45  ...             PayPal        Annually

[5 rows x 19 columns]
PS C:\Users\vikas\Downloads\AI Assist Coding>

```

Summary: This script cleans a customer dataset by handling missing values, removing duplicates, and standardizing text fields like city and gender. It also converts categorical data into numerical format using encoding techniques. The final cleaned dataset is saved and ready for further analysis or machine learning tasks.

Task 2:

Prompt: “Generate a Python script using Pandas to preprocess sales data by converting date columns to datetime, creating time-based features (month, year, day of week), normalizing the sales column, and detecting and handling outliers using the IQR method. Ensure the output dataset is ready for analysis.”

Code:

```
61 import pandas as pd
62 import numpy as np
63
64 # Load dataset
65 data = pd.read_csv("sales_data.csv")
66
67 # Display original data
68 print("Original Data:")
69 print(data.head())
70
71 # -----
72 # 1. Convert Date Column
73 # -----
74 if 'Date' in data.columns:
75     data['Date'] = pd.to_datetime(data['Date'], errors='coerce')
76
77 # -----
78 # 2. Create Time-Based Features
79 # -----
80 if 'Date' in data.columns:
81     data['Year'] = data['Date'].dt.year
82     data['Month'] = data['Date'].dt.month
83     data['DayOfWeek'] = data['Date'].dt.day_name()
```

```

87 # -----
88 if 'Sales' in data.columns:
89     data['Sales_Normalized'] = (data['Sales'] - data['Sales'].min()) / (data['Sales'].max() - data['Sales'].min())
90
91 # -----
92 # 4. Detect and Handle Outliers (IQR Method)
93 # -----
94 if 'Sales' in data.columns:
95     Q1 = data['Sales'].quantile(0.25)
96     Q3 = data['Sales'].quantile(0.75)
97     IQR = Q3 - Q1
98
99     lower_bound = Q1 - 1.5 * IQR
100    upper_bound = Q3 + 1.5 * IQR
101
102    # Replace outliers with median
103    median_value = data['Sales'].median()
104    data['Sales'] = np.where((data['Sales'] < lower_bound) | (data['Sales'] > upper_bound),
105                             median_value,
106                             data['Sales'])
107
108 # -----
109 # Final Output
110 # -----
111 print("\nProcessed Data:")
112 print(data.head())
113
114 # Save processed dataset
115 data.to_csv("processed_sales_data.csv", index=False)

```

Output:

```

[5 rows x 19 columns]
Original Data:
   Product_ID  Sale_Date Sales_Rep  ... Payment_Method Sales_Channel Region_and_Sales_Rep
0         1052  2023-02-03        Bob  ...          Cash         Online      North-Bob
1         1093  2023-04-21        Bob  ...          Cash         Retail        West-Bob
2         1015  2023-09-21       David  ...  Bank Transfer         Retail    South-David
3         1072  2023-08-24        Bob  ...  Credit Card         Retail    South-Bob
4         1061  2023-03-24     Charlie  ...  Credit Card         Online    East-Charlie

```

```

[5 rows x 14 columns]
Processed Data:
   Product_ID  Sale_Date Sales_Rep  ... Payment_Method Sales_Channel Region_and_Sales_Rep
0         1052  2023-02-03        Bob  ...          Cash         Online      North-Bob
1         1093  2023-04-21        Bob  ...          Cash         Retail        West-Bob
2         1015  2023-09-21       David  ...  Bank Transfer         Retail    South-David
3         1072  2023-08-24        Bob  ...  Credit Card         Retail    South-Bob
4         1061  2023-03-24     Charlie  ...  Credit Card         Online    East-Charlie

```

```

[5 rows x 14 columns]

```

```
[5 rows x 19 columns]
Original Data:
   Product_ID  Sale_Date  ... Sales_Channel Region_and_Sales_Rep
0         1052  2023-02-03  ...         Online         North-Bob
1         1093  2023-04-21  ...         Retail         West-Bob
2         1015  2023-09-21  ...         Retail        South-David
3         1072  2023-08-24  ...         Retail        South-Bob
4         1061  2023-03-24  ...         Online        East-Charlie
```

```
[5 rows x 14 columns]

Processed Data:
   Product_ID  Sale_Date  ... Sales_Channel Region_and_Sales_Rep
0         1052  2023-02-03  ...         Online         North-Bob
1         1093  2023-04-21  ...         Retail         West-Bob
2         1015  2023-09-21  ...         Retail        South-David
3         1072  2023-08-24  ...         Retail        South-Bob
4         1061  2023-03-24  ...         Online        East-Charlie
```

Summary: This script preprocesses sales data by converting date columns into datetime format and extracting useful time-based features like year, month, and day of the week. It normalizes the sales values for better analysis and applies the IQR method to detect and handle outliers. The final dataset is clean, structured, and ready for analytics or machine learning.

TASK 3:

Prompt: “Generate a Python script using Pandas and Scikit-learn to preprocess a healthcare dataset by handling missing values in blood pressure and cholesterol, encoding categorical variables, scaling numerical features, and splitting the dataset into training and testing sets for machine learning.”

Code:

```
119 # ----- Task 3 ----- #
120 import pandas as pd
121 from sklearn.preprocessing import MinMaxScaler, LabelEncoder
122 from sklearn.model_selection import train_test_split
123
124 # Load dataset
125 data = pd.read_csv("dirty_v3_path.csv")
126
127 # Display original data
128 print("Original Data:")
129 print(data.head())
130
131 # -----
132 # 1. Handle Missing Values
133 # -----
134
135 # Fill numerical columns with median
136 if 'blood_pressure' in data.columns:
137     data['blood_pressure'].fillna(data['blood_pressure'].median(), inplace=True)
138
139 if 'cholesterol' in data.columns:
140     data['cholesterol'].fillna(data['cholesterol'].median(), inplace=True)
141
```

```

145     label_encoders = {}
146
147     for col in data.select_dtypes(include=['object']).columns:
148         le = LabelEncoder()
149         data[col] = le.fit_transform(data[col].astype(str))
150         label_encoders[col] = le
151
152     # -----
153     # 3. Scale Numerical Features
154     # -----
155     scaler = MinMaxScaler()
156
157     num_cols = data.select_dtypes(include=['int64', 'float64']).columns
158     data[num_cols] = scaler.fit_transform(data[num_cols])
159
160     # -----
161     # 4. Split Dataset into Train & Test
162     # -----
163
164     # Assuming 'target' is the label column (change if needed)
165     target_column = data.columns[-1] # last column as target
166
167     X = data.drop(target_column, axis=1)
168     y = data[target_column]
169
170     X_train, X_test, y_train, y_test = train_test_split(
171         X, y, test_size=0.2, random_state=42
172     )

```

```

172     )
173
174     # -----
175     # Final Output
176     # -----
177     print("\nPreprocessed Data:")
178     print(data.head())
179
180     print("\nTraining Set Shape:", X_train.shape)
181     print("Testing Set Shape:", X_test.shape)

```

```
[5 rows x 14 columns]
Original Data:
   Age  Gender Medical Condition ... Sleep Hours random_notes noise_col
0  46.0   Male      Diabetes ...      6.05      lorem -137.057211
1  22.0   Male      Healthy ...      7.72      ipsum -11.230610
2  50.0   NaN      Asthma ...      4.82      ipsum  98.331195
3  57.0   NaN      Obesity ...      5.33      lorem  44.187175
4  66.0  Female      Hypertension ...      6.64      lorem  44.831426

[5 rows x 20 columns]

Preprocessed Data:
   Age  Gender Medical Condition ... Sleep Hours random_notes noise_col
0  0.455696      1      3 ...      0.509132      3  0.312605
1  0.151899      1      4 ...      0.699772      2  0.455579
2  0.506329      2      1 ...      0.368721      2  0.580072
3  0.594937      2      6 ...      0.426941      3  0.518549
4  0.708861      0      5 ...      0.576484      3  0.519281

[5 rows x 20 columns]

Training Set Shape: (24000, 19)
Testing Set Shape: (6000, 19)
```

Summary:

This script prepares a healthcare dataset by filling missing values in key medical fields like blood pressure and cholesterol. It encodes categorical features, scales numerical data for consistency, and splits the dataset into training and testing sets. The result is a clean and structured dataset ready for machine learning models.

TASK 4:

PROMPT: "Generate a Python script using Pandas and Scikit-learn to preprocess an employee salary dataset by handling missing values, removing salary outliers, standardizing department names, and applying label encoding to job location and department columns."

Code:

```
183 #-----task 4-----#
184 import pandas as pd
185 from sklearn.preprocessing import LabelEncoder
186
187 # Load dataset
188 data = pd.read_csv("Employee_Salary_Dataset.csv")
189
190 # Display original data
191 print("Original Data:")
192 print(data.head())
193
194 # -----
195 # 1. Handle Missing Values
196 # -----
197
198 # Fill numerical columns with median
199 num_cols = data.select_dtypes(include=['int64', 'float64']).columns
200 for col in num_cols:
201     data[col].fillna(data[col].median(), inplace=True)
```

```

204 cat_cols = data.select_dtypes(include=['object']).columns
205 for col in cat_cols:
206     data[col].fillna(data[col].mode()[0], inplace=True)
207
208 # -----
209 # 2. Detect & Remove Salary Outliers (IQR)
210 # -----
211
212 # Adjust column name if different
213 salary_col = 'salary' if 'salary' in data.columns else data.columns[-1]
214
215 Q1 = data[salary_col].quantile(0.25)
216 Q3 = data[salary_col].quantile(0.75)
217 IQR = Q3 - Q1
218
219 lower_bound = Q1 - 1.5 * IQR
220 upper_bound = Q3 + 1.5 * IQR
221
222 data = data[(data[salary_col] >= lower_bound) & (data[salary_col] <= upper_bound)]
223
224
225
227 if 'department' in data.columns:
228     data['department'] = data['department'].str.lower().str.strip()
229
230 # -----
231 # 4. Label Encoding
232 # -----
233 le = LabelEncoder()
234
235 for col in ['job_location', 'department']:
236     if col in data.columns:
237         data[col] = le.fit_transform(data[col])
238
239 # -----
240 # Final Output
241 # -----
242 print("\nProcessed Data:")
243 print(data.head())
244
245 # Save cleaned dataset
246 data.to_csv("processed_employee_salary.csv", index=False)

```

Output:

```

Original Data:
   ID  Experience_Years  Age  Gender  Salary
0    1                 5   28  Female  250000
1    2                 1   21   Male    50000
2    3                 3   23  Female  170000
3    4                 2   22   Male   25000
4    5                 1   17   Male   10000

Processed Data:
   ID  Experience_Years  Age  Gender  Salary
0    1                 5   28  Female  250000
1    2                 1   21   Male    50000
2    3                 3   23  Female  170000
3    4                 2   22   Male   25000
4    5                 1   17   Male   10000

```

PS C:\Users\vikas\Downloads\AI Assist Coding>

Summary:

This script preprocesses employee salary data by handling missing values and removing salary outliers using the IQR method. It standardizes department names for consistency and encodes categorical fields like job location and department. The final dataset is clean and structured for HR analytics and decision-making.